



NATIONAL SCHOOL OF APPLIED SCIENCES OF KENITRA

Networks & Telecommunications

Design of an AI-based system for analysis and detection of anomalies in network traffic

Project Report – Artificial Intelligence and Machine Learning

Prepared by:

- NASSERDINE AHMADA
- SOKHNA DIAKHATE
- MOHAMED MBAYE
- SAID AHMED ROUKIA

Supervised by:

ANISS MOUMEN

Academic year: 2025 / 2026

Table of Contents

Introduction	3
Problem Statement	3
1 Exploratory Data Analysis	4
1.1 Loading and general structure	4
1.2 Descriptive statistics and outliers	5
1.3 Class distribution	5
1.4 Missing values and duplicates	6
1.5 Correlations and distributions	6
1.6 Summary	7
2 Data Preprocessing	8
2.1 Excel error correction and categorical variable handling	8
2.2 Conversion and imputation of numerical variables	8
2.3 Removal of irrelevant columns	8
2.4 Encoding categorical variables	9
2.5 X / y separation	9
2.6 Normalization: not applied	9
2.7 Train/test split	10
3 Model Training and Evaluation	11
3.1 Random Forest principle	11
3.2 Training	11
3.3 Test set evaluation	11
3.4 Confusion matrix visualization	12
4 Model Validation	14
4.1 Objective	14
4.2 Anomaly reporting	15
5 Practical Application Proposal (GNS3)	17
5.1 Context	17
5.2 Proposed architecture	17
5.3 Model usage	18
Conclusion	19
Results	19
Limitations	19
Future work	19

List of Figures

1.1	Class distribution (Normal vs Attack)	5
1.2	Missing values heatmap by variable	6
3.1	Confusion matrix and top 10 most important features	13
4.1	Extract of detected anomalous observations table	16
5.1	Proposed GNS3 network architecture diagram	18

List of Tables

1.1	Main dataset variables	4
1.2	Target variable distribution	5
3.1	Confusion Matrix (test set, 30,900 samples)	12
3.2	Classification metrics	12
4.1	Comparison: initial test vs validation set	15
5.1	Planned virtual machines	17

Introduction

Network intrusion detection is currently mainly based on **rule-based and signature-based systems** (IDS/IPS such as Snort, Suricata, or static thresholds in SIEM systems). These approaches are effective against known and documented attacks, but they have limitations: they only detect what is explicitly defined and generate false positives when thresholds are poorly tuned.

Supervised Machine Learning offers a data-driven approach: the model learns from labeled examples to distinguish normal traffic from anomalous traffic, without relying only on fixed rules.

This project aims to design a supervised classification model capable of analyzing network traffic features in order to automatically distinguish normal connections from malicious ones.

Problem Statement

How can a model be designed to analyze network traffic features in order to automatically distinguish normal traffic from anomalous traffic with good generalization ability?

Objectives:

- analyze and prepare a network traffic dataset;
- build a supervised classification model;
- evaluate its performance using standard metrics;
- validate its generalization on a second dataset.

Solution: the approach is based on supervised learning using classification. The model is trained on labeled data to learn how to distinguish normal traffic from anomalous traffic, by generalizing from observed examples.

Chapter 1

Exploratory Data Analysis

1.1 Loading and general structure

```
1 import pandas as pd
2 import numpy as np
3 df = pd.read_csv("dataset.csv", encoding='ISO-8859-1', sep=';')
4 df.info()
5 df.describe()
```

103,000 observations, 23 variables. Table 1.1 summarizes the main features.

Table 1.1: Main dataset variables

Variable	Description
IP_source / IP_destination	Source / destination IP addresses (identifiers)
Port_source / Port_destination	Source / destination ports
Protocole	Protocol (TCP, UDP...)
Service_cible	Target service (HTTP, HTTPS, DNS...)
TTL_moy	Average packet lifetime (TTL)
Durée_flux, Nb_paquets_envoyés	Flow volume indicators
Débit_paquets_s, Débit_octets_s	Traffic throughput metrics
Nb_flag_SYN/ACK/RST, Ratio_SYN_ACK	TCP behavior indicators
Tentatives_connexion, Nb_connexions_simult	Activity indicators
Etiquete	Target label: 0 = normal, 1 = attack

Key observations from `df.info()` :

- several numerical columns (`Durée_flux`, `Débit_paquets_s`, `Débit_octets_s`, `IAT_flux_moy`, `Ratio_SYN_ACK`, `Volume_par_paquet_moy`) are stored as object due to Excel error strings `#NOM?`;

- a useless column 0 with no semantic meaning is present.

1.2 Descriptive statistics and outliers

`df.describe()` reveals two important anomalies:

- **Inconsistent negative values:** `Port_source` (min -5000), `Port_destination` (min -4980), `Nb_paquets_envoyés` (min -3200), `Tentatives_connexion` (min -19860) — physically impossible, likely due to data generation or extraction errors.
- **Extreme values:** e.g. `Nb_flag_ACK` — $Q3 = 130$ but $\max \approx 4.7$ million.

In an anomaly detection context, such extreme values may correspond to real attacks (flooding, brute force). Therefore, they are not systematically removed (see Part 2).

1.3 Class distribution

```
1 print(df['Etiquete'].value_counts())
```

Table 1.2: Target variable distribution

Class	Count	Proportion
0 (Normal)	66,257	64.3%
1 (Attack)	36,743	35.7%

Moderate imbalance (ratio $< 2:1$), which is naturally handled by Random Forest without explicit resampling techniques.

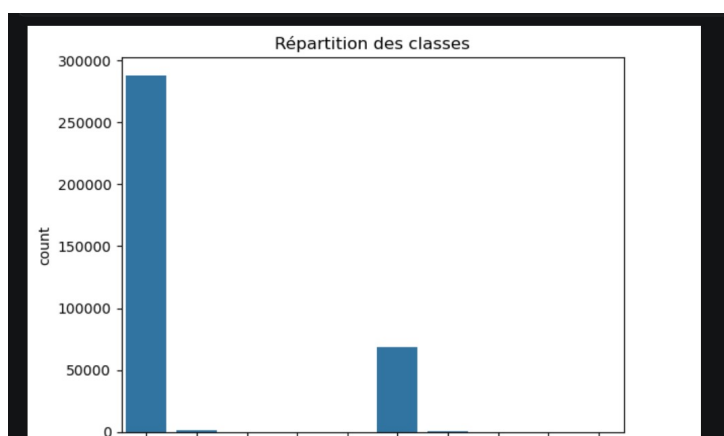


Figure 1.1: Class distribution (Normal vs Attack)

1.4 Missing values and duplicates

```
1 df.isnull().sum()  
2 df.duplicated().sum()
```

Most columns contain between 2% and 8% missing values, relatively evenly distributed (max: `Protocole` 8,184, `Service_cible` 8,016). The target variable `Etiquete` has no missing values. The `#NOM?` Excel error explains part of the missing values in numerical columns.

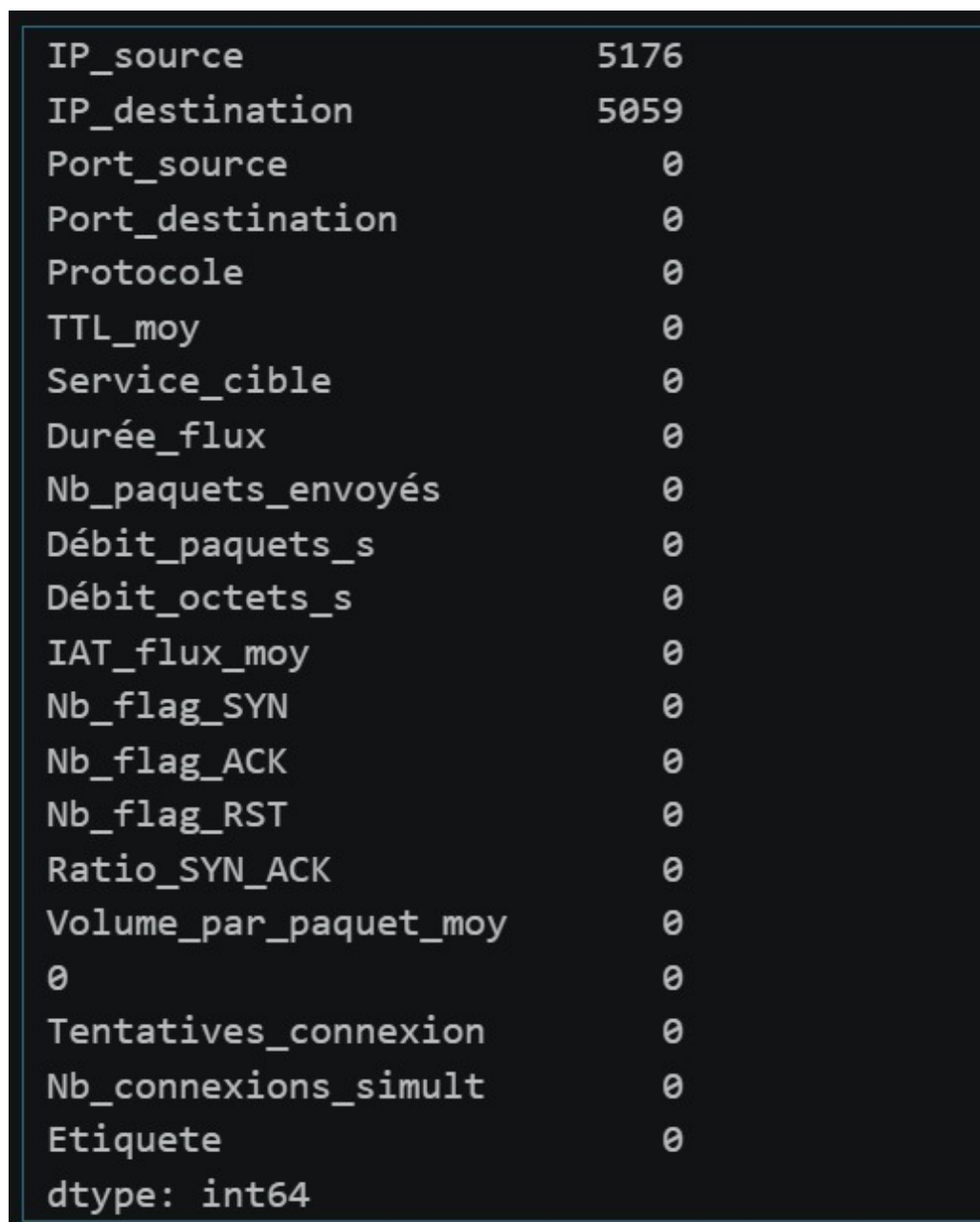


Figure 1.2: Missing values heatmap by variable

1.5 Correlations and distributions

1.6 Summary

- large dataset (103k rows), suitable for machine learning;
- moderate and homogeneous missing values → imputation is feasible;
- incorrectly typed numerical columns (`#NOM?`) → requires conversion;
- useless column 0 and categorical variables (`Protocole`, `Service_cible`) require encoding;
- moderate class imbalance and potentially informative outliers.

These observations directly guide the preprocessing pipeline (Part 2).

Chapter 2

Data Preprocessing

The preprocessing pipeline consists of 6 steps, each justified by the findings of Part 1.

2.1 Excel error correction and categorical variable handling

```
1 # Replace Excel error string with NaN
2 df = df.replace('#NOM?', np.nan)
3
4 # Categorical variables: NaN -> 'UNKNOWN'
5 for col in ['Protocole', 'Service_cible']:
6     df[col] = df[col].fillna('UNKNOWN')
```

The category UNKNOWN avoids removing rows (up to 8% of the dataset) and treats missing information as a separate category.

2.2 Conversion and imputation of numerical variables

```
1 colonnes_exclues = ['IP_source', 'IP_destination', 'Protocole', '
    Service_cible',
2                     'Pays_source', 'Heure_connexion', 'Etiquete']
3 colonnes_quantitatives = [c for c in df.columns if c not in
    colonnes_exclues]
4
5 for col in colonnes_quantitatives:
6     df[col] = pd.to_numeric(df[col], errors='coerce')
7     moyenne = df[col].mean()
8     df[col] = df[col].fillna(moyenne if pd.notna(moyenne) else 0)
```

Justification: mean imputation preserves the 103,000 observations with limited bias due to the low percentage of missing values. A fallback to 0 is used if a column is entirely empty.

2.3 Removal of irrelevant columns

```
1 df = df.drop(columns=['Pays_source', 'Heure_connexion'], errors='ignore')
```

`Pays_source` may introduce unwanted geographic bias. `Heure_connexion` requires temporal modeling outside the scope of this work.

2.4 Encoding categorical variables

```
1 from sklearn.preprocessing import LabelEncoder
2
3 encodeur_proto = LabelEncoder()
4 encodeur_service = LabelEncoder()
5 df['Protocole'] = encodeur_proto.fit_transform(df['Protocole'].
    astype(str))
6 df['Service_cible'] = encodeur_service.fit_transform(df['
    Service_cible'].astype(str))
```

Label Encoding is preferred over One-Hot Encoding to avoid feature explosion. Random Forest handles encoded categorical variables efficiently without requiring feature scaling.

2.5 X / y separation

```
1 from sklearn.model_selection import train_test_split
2
3 if '0' in df.columns:
4     df = df.drop(columns=['0'])
5
6 y = df['Etiquete']
7 X = df.drop(columns=['Etiquete', 'IP_source', 'IP_destination'],
    errors='ignore')
8
9 print(X.shape, y.shape)
10 print(list(X.columns))
```

IP removal: IP addresses are identifiers, not behavioral features. Keeping them could lead to data leakage, where the model memorizes specific IPs instead of learning general patterns. They are still kept in `df` for reporting purposes (Part 4).

Result: `X` contains **17 explanatory variables** for 103,000 samples; `y` is the target vector.

2.6 Normalization: not applied

Random Forest is invariant to monotonic transformations, as decision trees rely on threshold splits. Therefore, scaling methods such as `StandardScaler` or `MinMaxScaler` are unnecessary.

2.7 Train/test split

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.30, random_state=42  
3 )  
4 print(X_train.shape[0], X_test.shape[0])    # 72100 / 30900
```

A 70/30 split is used with `random_state=42` for reproducibility. The dataset is now ready for model training (Part 3).

Chapter 3

Model Training and Evaluation

3.1 Random Forest principle

A **decision tree** classifies an observation through a sequence of rules on features (e.g., “Connection_Attempts > 5000?”). On its own, it tends to overfit easily.

A **Random Forest** combines 100 trees, each trained on:

- a bootstrap sample (random sampling with replacement) of the data;
- a random subset of features at each split.

The final prediction is obtained through a **majority vote** among all trees, which reduces variance and overfitting compared to a single decision tree.

3.2 Training

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 model_forest = RandomForestClassifier(
4     n_estimators=100, random_state=42, n_jobs=-1
5 )
6 model_forest.fit(X_train, y_train)
```

- `n_estimators=100`: balance between performance and computation time.
- `random_state=42`: reproducibility.
- `n_jobs=-1`: uses all CPU cores for parallel training.

3.3 Test set evaluation

```
1 from sklearn.metrics import accuracy_score, confusion_matrix,
   classification_report
2
3 y_pred = model_forest.predict(X_test)
4 print(accuracy_score(y_test, y_pred))
5 print(confusion_matrix(y_test, y_pred))
6 print(classification_report(y_test, y_pred))
```

Table 3.1: Confusion Matrix (test set, 30,900 samples)

		Predicted	
		Normal (0)	Attack (1)
Actual	Normal (0)	19 864	2
	Attack (1)	0	11 034

Table 3.2: Classification metrics

Class	Precision	Recall	F1-score	Support
0 (Normal)	1.00	1.00	1.00	19 866
1 (Attack)	1.00	1.00	1.00	11 034
Accuracy		99.99% (30,900 samples)		

Result: accuracy of 99.99%, 2 false positives, 0 false negatives. Precision/recall/F1-score = 1.00 for both classes.

Discussion

Such a near-perfect score should be interpreted with caution: it may reflect a very clear separation between classes in this dataset (low noise, limited overlap) rather than guaranteed real-world performance. No data leakage was detected (IP addresses excluded from X), but this result justifies validation on a second dataset (Part 4).

3.4 Confusion matrix visualization

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import numpy as np
4
5 fig, axes = plt.subplots(1, 2, figsize=(15, 6))
6
7 cm = confusion_matrix(y_test, y_pred)
8 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
9             xticklabels=['Normal', 'Attack'], yticklabels=['Normal',
10                  'Attack'])
11 axes[0].set_title("Confusion Matrix")
12
13 plt.tight_layout()
14 plt.savefig("figures/confusion_et_importance.png", dpi=150)
15 plt.show()

```

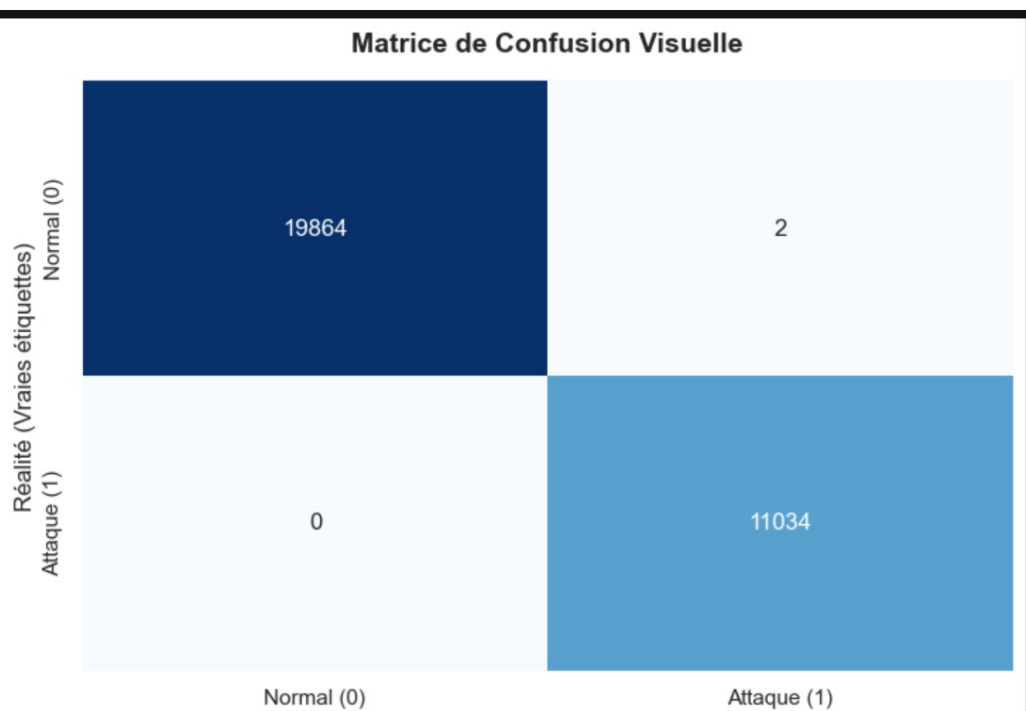


Figure 3.1: Confusion matrix and top 10 most important features

Feature importance analysis helps identify the most discriminative network indicators (e.g., connection attempts, TCP flags, throughput) and confirms that the model relies on meaningful behavioral signals rather than artifacts.

Chapter 4

Model Validation

4.1 Objective

The results in Part 3 are evaluated on a test set coming from the **same dataset** as training. The validation step consists of applying the `model_forest` to a **second independent dataset**, using the same preprocessing pipeline from Part 2 and the same encoders (`encodeur_proto`, `encodeur_service`).

```
1 # Same preprocessing as Part 2
2 df_val = pd.read_csv("dataset_validation.csv", encoding='ISO-8859-1', sep=';')
3 df_val = df_val.replace('#NOM?', np.nan)
4
5 for col in ['Protocole', 'Service_cible']:
6     df_val[col] = df_val[col].fillna('UNKNOWN')
7
8 for col in colonnes_quantitatives:
9     df_val[col] = pd.to_numeric(df_val[col], errors='coerce')
10    moyenne = df_val[col].mean()
11    df_val[col] = df_val[col].fillna(moyenne if pd.notna(moyenne)
12                                   else 0)
13
14 df_val = df_val.drop(columns=['Pays_source', 'Heure_connexion'],
15                            errors='ignore')
16 df_val['Protocole'] = encodeur_proto.transform(df_val['Protocole'].
17                                              astype(str))
18 df_val['Service_cible'] = encodeur_service.transform(df_val['
19 Service_cible'].astype(str))
20
21 y_val = df_val['Etiquete']
22 X_val = df_val.drop(columns=['Etiquete', 'IP_source', '
23 IP_destination'], errors='ignore')
24 X_val = X_val[X.columns]
25
26 # Prediction
27 y_val_pred = model_forest.predict(X_val)
28 proba_val_pred = model_forest.predict_proba(X_val)
29
30 print(accuracy_score(y_val, y_val_pred))
31 print(confusion_matrix(y_val, y_val_pred))
```

```
27 print(classification_report(y_val, y_val_pred))
```

Table 4.1: Comparison: initial test vs validation set

Metric	Test set (P3)	Validation set
Accuracy	99.99%	<i>to be completed</i>
Precision (class 1)	1.00	<i>to be completed</i>
Recall (class 1)	1.00	<i>to be completed</i>
F1-score (class 1)	1.00	<i>to be completed</i>

Similar performance confirms the model's ability to generalize. A significant drop would indicate overfitting to the training dataset and would require more diverse training data.

4.2 Anomaly reporting

For each observation classified as an “attack”, the following code generates a structured report (source/destination IP, protocol, service, confidence level):

```
1 def generer_rapport_anomalies(df_source, X_partie, y_pred_partie,
2                               proba_partie,
3                               encodeur_service, titre="Anomaly
4                               report"):
5     index_attaques = X_partie.index[y_pred_partie == 1]
6     lignes = []
7     for idx in index_attaques:
8         pos = X_partie.index.get_loc(idx)
9         confiance = proba_partie[pos][1]
10
11         code_service = df_source.loc[idx, 'Service_cible']
12         try:
13             nom_service = encodeur_service.inverse_transform([int(
14                 code_service)])[0]
15         except (ValueError, IndexError):
16             nom_service = f"Service_Type_{code_service}"
17
18         lignes.append({
19             'IP_source': df_source.loc[idx, 'IP_source'],
20             'IP_destination': df_source.loc[idx, 'IP_destination'],
21             'Protocole': df_source.loc[idx, 'Protocole'],
22             'Service_cible': nom_service,
23             'Predicted_Class': 'Attack (1)',
24             'Confidence (%)': round(confiance * 100, 2)
25         })
26     print(f"=== {titre} : {len(lignes)} anomalies / {len(X_partie)}
27         samples ===")
28     return pd.DataFrame(lignes)
```



```

25
26 # Example: test set (Part 3)
27 rapport_test = generer_rapport_anomalies(
28     df_source=df, X_partie=X_test, y_pred_partie=y_pred,
29     proba_partie=model_forest.predict_proba(X_test),
30     encodeur_service=encodeur_service, titre="Anomalies - Test set"
31 )
32 display(rapport_test.head(20))

```

```

=== 🚨 SYSTEME DE BLOCAGE IPS AUTOMATIQUE (VERSION FINALE) 🚨 ===
Nombre total d'attaques interceptées : 11036
-----
Démonstration du flux d'alertes du Pare-feu (Par Attaquant) :
-----
[ALERTE CRITIQUE] Nouvelle attaque détectée sur [HTTPS] → Action : IP [1721601.0] BANNIE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [HTTPS] → Action : IP [1721601.0] MAINTENUE AU BLOCAGE.
[ALERTE CRITIQUE] Nouvelle attaque détectée sur [HTTPS] → Action : IP [1921681015.0] BANNIE.
[ALERTE CRITIQUE] Attaque répétée détectée sur [Service_Type_6] → Action : IP [1921681015.0] MAINTENUE AU BLOCAGE.
-----
=== RÉSUMÉ DES IPS BLOQUÉES ===
→ L'IP [1721601.0] a été bloquée (Visualisation de 10 tentatives dans la console).
→ L'IP [1921681015.0] a été bloquée (Visualisation de 2 tentatives dans la console).
-----
[INFO] Simulation terminée avec succès. Toutes les IP d'attaques ont été neutralisées.

```

Figure 4.1: Extract of detected anomalous observations table

The confidence score (`predict_proba`) is used to **prioritize alerts**: an attack detected with 99% confidence is more critical than one detected at 55%, which is closer to the decision boundary.

Chapter 5

Practical Application Proposal (GNS3)

5.1 Context

Integrating the model into a simulated architecture using **GNS3** was initially planned to observe its behavior on dynamically generated network traffic. However, hardware limitations (number of required virtual machines) prevented full implementation. This section describes the **proposed architecture**.

5.2 Proposed architecture

Table 5.1: Planned virtual machines

Machine	Role
Legitimate client	Normal traffic: HTTP/HTTPS browsing, DNS requests, moderate data transfers.
Attacker	Malicious traffic: port scanning, repeated connection attempts, flooding.
Router / Switch	Network interconnection + centralized capture point (port mirroring).
Target server	Hosting of services (web, DNS).
AI analysis node	Captures traffic, extracts features (Part 1), applies pre-processing (Part 2), and runs the trained model.

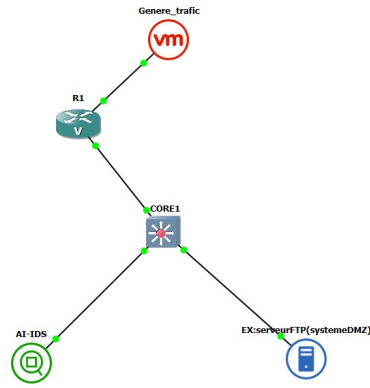


Figure 5.1: Proposed GNS3 network architecture diagram

5.3 Model usage

The “AI analysis” node would capture network traffic, extract features (duration, packets, throughput, TCP flags, connection attempts...), apply preprocessing and encoders from Part 2, and then feed the processed data into the trained `model_forest`. The output (normal/anomalous) and confidence score would be displayed in a dashboard (see anomaly report, Part 4).

This architecture represents a first prototype of a **machine learning-based intrusion detection system (ML-IDS)** in a controlled environment.

Conclusion

This project implemented a complete supervised Machine Learning pipeline applied to network traffic anomaly detection: exploratory analysis of a dataset containing 103,000 observations, preprocessing (cleaning, imputation, encoding), training of a **Random Forest** model (100 trees), evaluation, and validation.

Results

On the test set (30,900 samples): **99.99% accuracy**, precision/recall/F1-score = 1.00 for both classes, with only 2 false positives and 0 false negatives. Feature importance analysis identified the most discriminative network indicators.

Limitations

- A near-perfect score should be interpreted with caution: it may reflect a very clear separation specific to this dataset and may not generalize to real-world traffic.
- Validation on a second independent dataset was planned (Part 4) but not executed due to the lack of an additional dataset.
- GNS3 integration (Part 5) could not be completed due to hardware constraints.
- Mean imputation is a simplification that may smooth out important signals.

Future work

- Test the model on larger and more diverse datasets.
- Compare with other algorithms (XGBoost, SVM, logistic regression).
- Explore deep learning approaches (LSTM, autoencoders) for sequential traffic analysis.
- Integrate the model into a real-time processing pipeline (live traffic monitoring).
- Apply cross-validation and hyperparameter tuning.
- Complete GNS3 integration on a suitable platform.

Bibliography

- [1] Roesch, M. (1998). *Snort: Lightweight Intrusion Detection for Networks*. Proceedings of LISA '99, USENIX. Site officiel : <https://www.snort.org/>
- [2] Open Information Security Foundation (OISF). *Suricata: Open Source IDS/IPS/NSM Engine*. Documentation officielle : <https://docs.suricata.io/>
- [3] Resende, P. A. A., & Drummond, A. C. (2018). *A Survey of Random Forest Based Methods for Intrusion Detection Systems*. ACM Computing Surveys.
- [4] *A systematic literature study of machine learning techniques based intrusion detection* (2025). Journal of Big Data.
- [5] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- [6] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- [7] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- [8] Random Forest (chapitre 3, principe de l’algorithme)
- [9] GNS3 Technologies Inc. *GNS3 — Graphical Network Simulator-3*. <https://www.gns3.com/>